

Protocol Completion of a Robust C++ Virtual Switch

Jasper Haag* Yoichi Hirai* Simon Hudon*
Awais Masood Gregory Malecha Gordon Stewart

August 1st, 2024

Abstract

This report presents protocol completion, a new solution to robust safety over shared-memory protocols such as VIRTIO. Protocol completion provides a generic specification for an application that implements a protocol with clients that might not always follow the protocol. Unlike approaches to robust safety that duplicate proofs, protocol completion lets one prove a system only once, deriving functional and safety guarantees independently on top against both compliant and adversarial clients. As evaluation, we apply protocol completion to verification of the dataplane of a virtual Ethernet switch running on BlueRock Security, Inc.’s microkernel-based operating system.

1 Introduction

BlueRock Security, Inc. is building a formally verified hypervisor-based security solution for runtime monitoring of cloud and bare-metal workloads (Figure 1). A core component of BlueRock’s userland is the vSwitch, a virtual Ethernet switch implementing the VIRTIO Network Device protocol [14]. Like all VIRTIO Devices, VIRTIO Network Devices such as the vSwitch are implemented in terms of a VIRTIO shared-memory protocol for bulk data transport. The eponymously named virtqueue implements this protocol and enables data transfers between guest drivers and virtual devices (e.g., serial, network, storage). Each virtqueue contains a descriptor buffer pointing to payloads in shared memory and two ring queues for sharing descriptors between parties: the `available` ring for sending descriptors from drivers to devices and the `used` ring for sending descriptors back from devices to drivers. Descriptors can be chained in the manner of scatter-gather buffers, and chains of descriptors can share both descriptors and payloads.

A virtual device like vSwitch may send the same packet to multiple guests but does not maintain local copies of the nonheader parts of packet payloads. Instead, it reads the payload buffer multiple times, once per destination. In the case of multicast packets, there may be many such destinations, one per port in a VLAN. Reading the payload more than once reduces copying but means that the virtual device may observe more behaviors of the driver, including buggy and/or adversarial ones. A buggy driver could forget to issue a memory fence, for example, leading to incorrectly ordered memory observations. Alternatively, a malicious driver could overwrite the payload memory while the device is sending a packet, resulting in one destination of a multicast receiving a version of the packet that is different from another.

The possibility of bad behaviors at the shared-memory interface of the protocol complicates specification and verification. For clients that follow the protocol, the implementation should behave according to protocol. For vSwitch, this means never delivering packets to incorrect destination

*The first three authors contributed equally and are listed alphabetically.

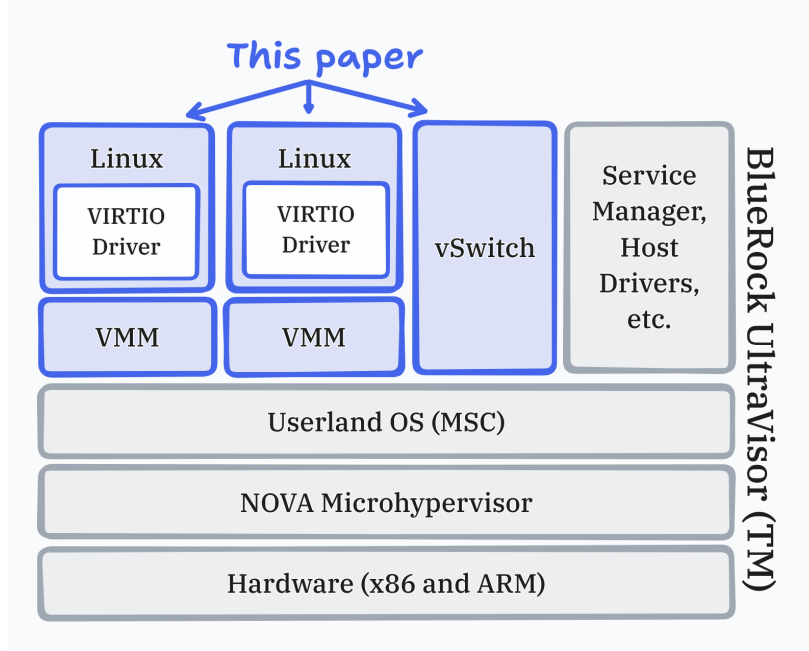


Figure 1: The BlueRock ULTRA™ virtualization stack. On top of x86 and ARM, the NOVA Microhypervisor supports a userland OS that runs Linux guests on top of virtual machine monitors (VMMs). The focus of this report, vSwitch, provides VIRTIO-based virtual Ethernet switching among guests.

ports and never violating VLAN isolation. Against guests that deviate from the protocol in possibly adversarial ways, the implementation should continue to satisfy basic safety guarantees. For vSwitch, this means correctly routing packets from well behaved guests in the presence of the ill behaved ones and enforcing VLAN isolation even for adversarial guests. Achieving this kind of security property in the presence of adversaries is called *robust safety* [5].

Traditional approaches to proving robust safety of shared-memory protocols solve the problem by proving the same system twice: once with respect to protocol-compliant clients and a second time with respect to arbitrary (potentially adversarial) ones. In concurrent separation logic, this second proof is typically formulated by an invariant over the shared resources that leaves the resources unconstrained (e.g., an array with existentially quantified values) [12]. Since the state of the shared resources is arbitrary, the specification one proves in this adversarial case is often weaker than the spec that is provable in the protocol-compliant case.

The problems of this invariant-based approach are twofold. First, one must prove the same system twice, with respect to two different specifications. This extra proof effort should not be underestimated; for systems of thousands of lines of code, requiring two proofs may make a verification effort infeasible. Second, traditional robust safety presumes one can determine *a priori* which clients are adversarial and which are not. This distinction is useful in determining which proof applies to which client but is unrealistic for systems like BlueRock’s hypervisor (Figure 1). In practice, a single client may be both adversarial and compliant over the lifetime of the protocol. Strictly speaking, the attack model of our hypervisor assumes that all guest code is adversarial. We nevertheless want a guarantee that vSwitch follows the VIRTIO protocol when guests do, and some guests may behave according to protocol for some time, only misbehaving after hours or days. Achieving this guarantee in traditional robust safety is difficult when we cannot label guests *a priori*.

Contributions. The main contribution of this report is *protocol completion* (Section 2), a new technique that solves robust safety without requiring multiple proofs of a system and without labeling clients. Protocol completion is a generic technique that works on any shared-memory protocol. The main idea is to close the semantics over all the client misbehaviors potentially observed by the implementation of the protocol, allowing the implementation to behave according to any of its observations. This results in a specification that implies functional correctness for well-behaved clients as well as traditional robust safety guarantees for ill-behaved ones.

Tracking observations rather than shared state and allowing behavior according to any observation yield a semantics that injects nondeterminism precisely at the points at which clients are observed to misbehave. This constrained nondeterminism arising from client misbehavior enables optimizations like read coalescing (an implementation may assume that two reads to a read-only location yield the same value) but does not poison the semantics in the way that undefined behavior does in some programming languages. At the same time, for a round-based shared-memory protocol like VIRTIO, it is important to consider when observations are no longer live and must be retired. Observation retirement limits the temporal consequences of client misbehavior to the round in which the client misbehaved, producing a kind of temporal isolation of misbehaviors and stronger dynamic guarantees.

We introduce protocol completion on a small example (VIRTIO₀, Section 2) and then evaluate it on the dataplane of vSwitch (Section 4), a virtual Ethernet switch implemented on top of BlueRock’s hypervisor (Figure 1). In Section 8, we evaluate the vSwitch itself (e.g., in terms of latency and throughput) and discuss specification and proof effort.

Availability. The BRICK C++ semantics on which our verification is based is available online at [13]. BlueRock Security, Inc. is in the process of preparing open access or access with an academic license to our verification results.

Limitations. The formal development (but not the C++ code) associated with this report does not yet account for weak memory. Formalizing weak memory is in scope of future work but is complicated by choice of hardware and architectural details. Second, our verification of the vSwitch dataplane (Section 4) relies on a specified but unproved C++ implementation of the VIRTIO protocol. Our work on VIRTIO specification led to interesting refinements of our model that are being incorporated into an on-going proof of the library. The proof of the dataplane depends on assumptions documented in file `README-admits.md` in the formal development and briefly described in Section 7.3. The proof of the control plane and a strengthening of the “no direct flows” property are in progress. More detail is in Section 7.

2 Protocol Completion

Completing a formal system means closing it under some property of interest. Taking the transitive completion or closure of a graph, for example, means adding edges between nodes that were previously connected only by multi-hop paths. The transitive closure of the step relation of a semantics similarly adds a rule directly connecting states that were originally reachable only under multiple steps.

Analogously, protocol completion adds rules to the specification of a shared-memory system to permit behaviors beyond the original protocol. The goal is a semantics that simultaneously implies functional correctness and robust safety (Figure 2). For the strongest guarantees, it is also important that the semantics isolate misbehaviors both spatially and temporally. Spatial

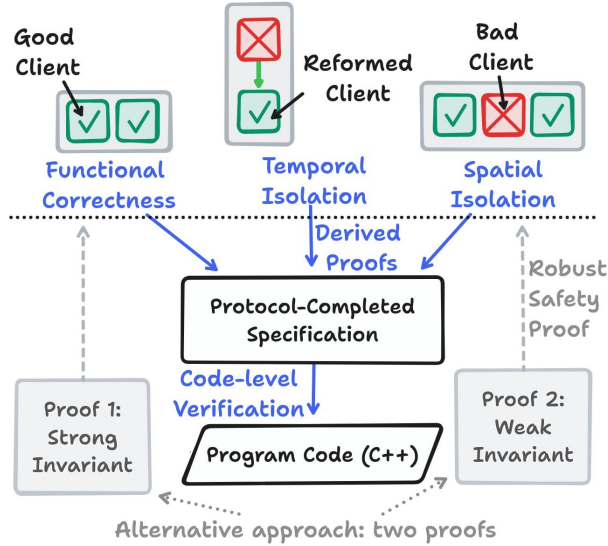


Figure 2: Goals of protocol completion. From a single protocol-completed specification, derive proofs of functional correctness and robust safety in the presence of bad clients. Bad behavior is isolated both temporally and spatially. Alternative approaches use two invariants and two separate proofs.

isolation means that in a multiclient system like vSwitch, one guest’s misbehavior does not poison vSwitch’s guarantees to another well-behaved client. By temporal isolation, we mean that a client that previously misbehaved (e.g., in a prior round of the protocol) may redeem itself by behaving according to protocol in a future round.

Like undefined behavior in programming languages such as C and C++, the constrained non-determinism introduced by protocol completion enables an implementation of a protocol to optimize under the assumption of good behavior. If two live VIRTIO descriptor chains alias the same payload memory for example, a valid implementation could decide to read the payload memory only once. A difference from undefined behavior (UB) in programming languages is that for protocol completion the effect of the UB is limited in scope both spatially and temporally. While a compiler is free to optimize a program with UB to `skip`, a protocol-completed semantics should exploit bad behavior only locally. In the case of read coalescing, a misbehaving client could violate the read-only assumption by writing the aliased descriptor between the two reads. The implementation may then send two different packets from the bad guest but should not use the bad guest’s misbehavior as an excuse to misdeliver an unrelated client’s packets.

2.1 Protocol Completion By Example

The **three guiding principles** of protocol completion are: 1) Specify observations, not shared state; 2) Allow behavior with respect to any legal observation; 3) Retire observations when the protocol demands it. We illustrate these principles and some additional details of protocol completion with reference to the the shared-memory protocol of Figure 3, which models some core aspects of VIRTIO.

In the figure, time goes from left to right. The guest driver at the top of the figure, which we consider the potentially adversarial client, begins by transferring a region of shared memory (middle lane) to the verified device (bottom lane). The device gains access to the shared memory in some way, e.g., by waking from a semaphore or reading a previously shared atomic field. The device then

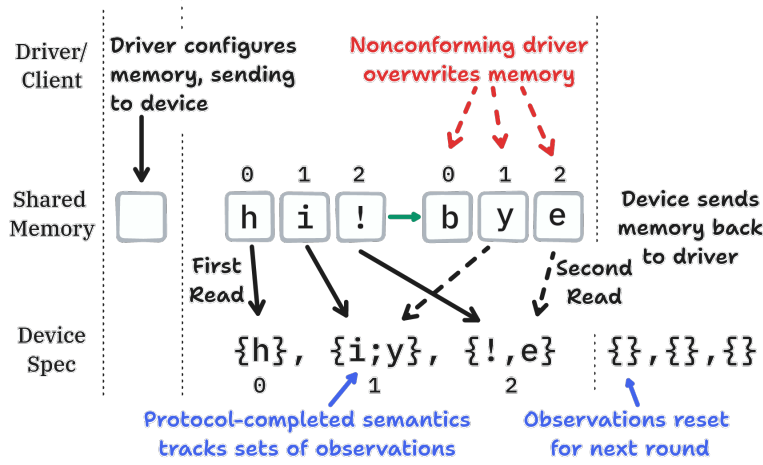


Figure 3: Protocol completion of VIRTIO₀, a simplified VIRTIO-like shared-memory protocol. The Driver initiates the protocol by configuring a shared memory region and sending it to the Device. The Device reads parts of the memory region twice, allowing it to observe illegal effects from the Driver. The Device’s protocol-completed specification models sets of observations for each field, allowing behavior with respect to any observation but resetting observations to $\{\}$ at the end of the round.

reads the contents of the buffer and forwards it to an I/O device or to a network port. In the figure, we show the case in which the device reads part of the packet (e.g., the payload) multiple times. This models the multicast case of VIRTIO. Once the device has processed the packet, it sends the packet’s backing memory to the driver for reuse and resets.

Specify observations, not shared state. Protocol-completed specifications focus not on the explicit state of shared memory but on the observations a program performs on that memory. In Figure 3, these observations are represented as sets of values associated with each index of the shared buffer, which in an operational model we might characterize as:

Definition $\text{obs} := \text{vec size } (\text{gset } N).$

This is essentially a power-set transformation of a naive semantics of the protocol. The step relation of the model is likewise updated to add additional observations on each read operation.

In the example, the device reads the buffer once, learning observation `''hi!''`. If the device re-reads the buffer as part of a multicast operation in parallel with an adversarial driver overwriting the buffer, the device may observe an entirely different string `''bye''`. Protocol completion says that the device may behave according to either string, and in fact, according to any string constructible from the read characters. This specification may overapproximate the actual behavior of the device, but only in cases in which the driver violated the protocol. When the driver obeys the protocol, the device is guaranteed to observe the same string `''hi!''` twice, tightening its specification to the singleton set $\{\text{''hi!''}\}$.

In traditional robust safety, one might model Figure 3 by an invariant holding ownership of the shared memory, along with tokens enforcing the sharing protocol:

$\text{inv } (\text{token} \vee (\exists l, pa \mapsto \text{vecR Tchar size } l))$

The driver can establish the invariant with a vector of `size` characters at a shared physical address `pa`, sending the invariant and a new exclusive `token` to the device. The device can remove the ownership of the shared memory from the invariant upon receipt of the token from the driver (by placing the token in the invariant in exchange for the memory). When memory is shared across processes, the logic must represent the memory objectively, i.e., in terms of physical bytes rather than virtual memory or points-to facts indexed by the current process’s page table. This invariant is suitable for cases in which the driver is assumed or proved to transfer the memory correctly but not to follow the protocol. A second proof against a second invariant would be required for the case in which the driver is assumed to follow a strict protocol on the shared-memory contents *l*.

Allow behavior with respect to any legal observation. Implicit in `VIRTIO0` is the idea that the device is permitted to behave according to any legal observation it made on the shared state. Closing a protocol by the client’s nondeterminism in this way is important for optimizations, as discussed above. A natural question is whether the added nondeterminism makes the specification of the device weaker than necessary, e.g., does it permit implementations that violate expected guarantees like the fact that one client’s buggy behavior is isolated from protocol compliant guests.

We address this question formally in Section 6 by proving that our protocol completion semantics satisfies inductive invariants that imply clear isolation properties. In particular, we show that the vSwitch model permits “no direct flows”: transactions are never configured to send bytes from a source port in a VLAN that differs from the VLAN of the destination port of a given packet.

Retire observations when the protocol demands it. At the end of a phase of the protocol, the device’s completion semantics resets stale observations to the empty set. This step ensures that the device re-reads the buffer when the guest sends the next string. In separation logic, we use a combination of persistent predicates and generation number tokens encoded in ghost state (Section 4.3) to abstractly represent this kind of observation reclamation. Persistent observation predicates are usable only in combination with reclaimable generation numbers.

Determining when observations become unusable is straightforward in the case of Figure 3 but becomes more subtle for real-world protocols. Difficulties arise primarily because of indirection and batching (Figure 4). Unlike `VIRTIO0`, `VIRTIO` implements a scatter-gather protocol in which chains of descriptors are transferred between parties. Each descriptor uses a pointer and length field to indirect into a shared region of payload memory. Descriptors have `next` fields to enable descriptor chaining. Multiple descriptors can be sent in a single round (batching).

Indirection and descriptor chaining introduce the possibility of payload and descriptor aliasing, as in the Figure 4. The driver constructs a chain of descriptors, e.g., `[d1,d2]`, then sends the chain by writing the root of the chain `d1` in the available ring. The driver could send a second chain `[d3,d2]` in the same round, only notifying the device after both chains were constructed and placed in the available ring. Chains `[d1,d2]` and `[d3,d2]` share descriptor `d2`. Descriptors `d2` and `d3` alias the same chunk of memory, initially containing ‘i’. In such a protocol, which closely models actual `VIRTIO`, the device would write descriptor roots to a “used ring” to send descriptor chains back to the driver once the device is done processing them.

Aliasing is an issue because it complicates ownership retirement. Consider a driver that receives chain `[d1,d2]` and then chain `[d3,d2]` but processes only the first chain `[d1,d2]`. If the driver maintains its ownership of both chains, i.e., it has not placed either chain in the used ring, then protocol completion semantics justifies a driver optimization that re-uses observations on shared descriptor `d2` when processing the second chain `[d3,d2]`. The situation is less clear when the driver retires `[d1,d2]` before processing `[d3,d2]`. In this case, members of the `VIRTIO` standards

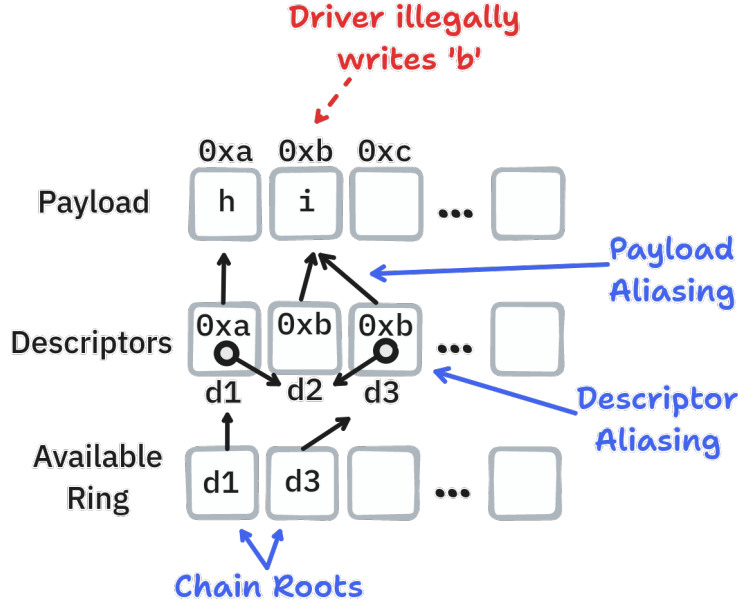


Figure 4: VIRTIO₀ with indirection and interference

committee clarified [1] that the unit of ownership is the descriptor, not the chain, and that the driver owns `d2` as long as it maintains an alias. The driver is therefore permitted to reuse its reads of `d2` when walking chain `[d3, d2]`. A similar rationale justifies the re-use of reads of payload memory, such as of cell β . If the driver illegally interferes by writing `'b'` to β between the device's walks of `[d1, d2]` and `[d3, d2]`, then protocol completion semantics says the device is justified in reading both chains as "hi". This semantics explicitly allows the optimization in which the driver reuses its prior observation of β when processing the second chain.

3 Extending to vSwitch

In this section and the next, we extend protocol completion to the dataplane of vSwitch, BlueRock's virtual Ethernet switch. We explain the vSwitch operational model first, which is similar to but more complex than VIRTIO₀ (Figure 3). Then (Section 4) we describe the C++ implementation of vSwitch and explain how we specify the connection between models and C++ code. Section 5 shows a specification and proof synthesis method called dataclasses that we use to generate parts of our proofs of network header parsing and serialization. Section 6 proves that our protocol-completed vSwitch model refines a high-level model with clear security guarantees. Although the focus of this report is the technique of protocol completion rather than empirical results, we do some basic benchmarking of vSwitch in Section 8 which demonstrates that vSwitch is industry-caliber software.

vSwitch Model

Figure 5 gives the basic structure of our vSwitch model.

We formalize the vSwitch model as a small-step operational semantics in Coq. The main state type `State.t` contains state of the ports (`port_map`), the network forwarding table (`fwd_tbl`) and

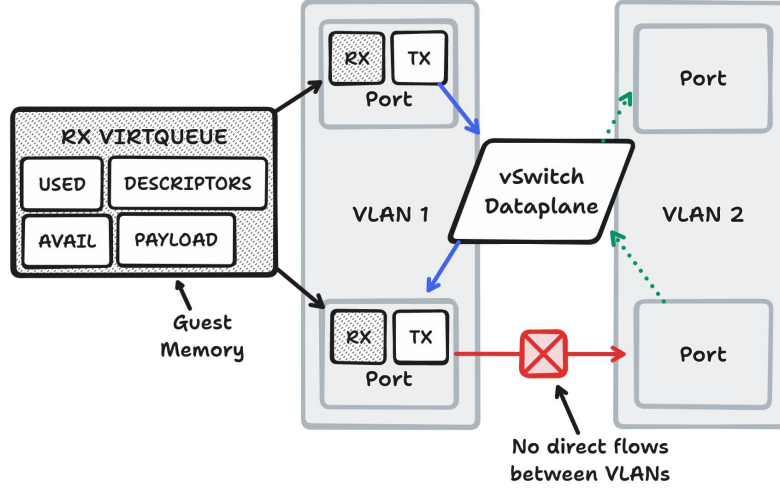


Figure 5: Structure of the vSwitch dataplane and its operational model. The dataplane reads packets from VIRTIO transmit buffers (Section 3.1) and routes them based on Ethernet destination MAC to other ports in the same VLAN. Later in Section 6, we derive a security property on the model, that vSwitch never directly forwards packets across VLANs. (This property does not preclude indirect flows.)

the current in-flight transactions:

```

NES.Begin State.
  Record t := {
    port_map : gmap port_id port
    ; fwd_tbl :
      gmap vlan_id (gmap MAC.t port_id)
    ; transactions : list transaction }.
  ...
  Definition initial : t :=
    { | port_map := ∅
      ; fwd_tbl := ∅
      ; transactions := [] | }.
NES.End State.

```

Each port is modeled as a pair of VIRTIO queues (virtqueues) and events from the small-step operational semantics of virtqueues are injected into the vSwitch trace. Given that the vSwitch is only exposed to adversarial behaviors through virtqueues, protocol-completing the small-step operational semantics for the latter is sufficient to protocol-complete the entire vSwitch model.

3.1 Protocol Completed VIRTIO Queue Model

The VIRTIO Queue protocol manages 3 shared memory regions - where each is writable by either the Driver or Device, but not both: the descriptor table (Driver W), the available ring (Driver W) and the used ring (Device W). The descriptor table contains scatter-gather chains which indirect into Driver memory for payloads. The available ring transfers the root descriptors of chains from the Driver to the Device. The used ring transfers the root descriptors of chains from the Device to the Driver.

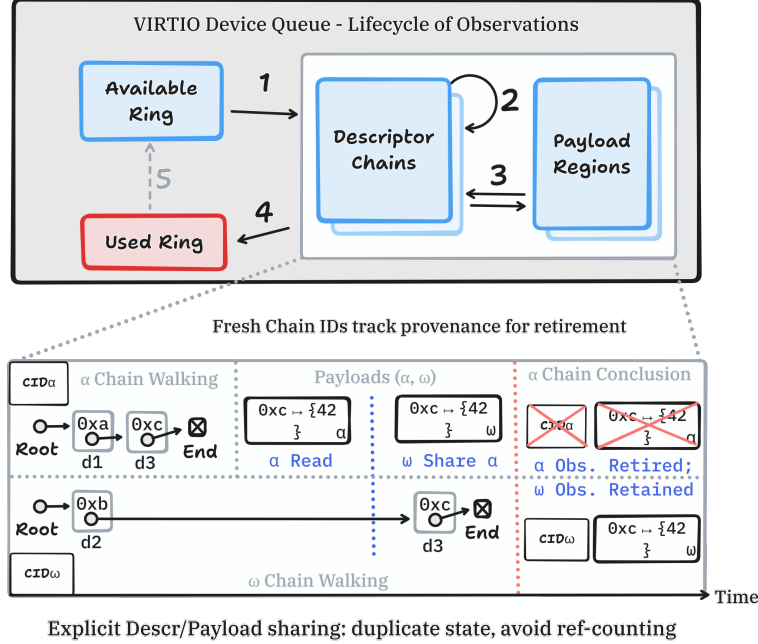


Figure 6: Legal observations of avail-ring entries justify building new descriptor chains (1); chain structure can be extended through reads or sharing (2); chains justify read/write access to payloads (3); chains can be batched for return to the driver via writes to the used-ring (4); returning a batch of chains derived from some avail-ring entries retires other chains derived from those entries (5).

3.1.1 Tracking VIRTIO Observations

Like VIRTIO_0 , the full VIRTIO operational model tracks legal observations for each accessible location under the protocol as opposed to directly tracking the state of the shared memory backing them. In the face of an adversarial or buggy Driver, this can result in multiple value being observed for the same location. For example, a descriptor might end up with multiple next links or an available ring entry might end up with multiple descriptor roots. In the case of descriptor metadata and payload values, however, the full VIRTIO model differs from VIRTIO_0 by duplicating state and exposing explicit sharing rules. This strategy proves useful when characterizing observation retirement (Section 3.1.3) - although it requires the use of fresh Chain IDs to track the provenance of observations (cf. Figure 6).

3.1.2 Nondeterministically Using VIRTIO Observations

The Device can leverage the nondeterminism of observations in three main places: (1) available ring entries, (2) descriptor address/length/flags fields and (3) the descriptor next field.

If a Device observes multiple values for an entry in the available ring that justifies creating a new chain rooted at each of the observed values. Any of these chains may be used for subsequent descriptor metadata/payload steps - although all of these chains must be retired simultaneously in order to ensure temporal isolation (cf. Section 3.1.3).

If a Device observes multiple values for the address/length/flags fields in a descriptor that justifies operating based on any of those values. For example, if one value of the flags indicates that the payload is readable and another value indicates that it is writable, then the Device is

permitted to both read and write from the payload memory. Similarly, if multiple address or length fields are observed then (potentially overlapping) ranges of guest addresses become accessible for reads/writes.

If a device observes multiple values for the next field in a descriptor then that justifies operating based on any of those values. However, once a Device commits to a particular non-deterministic choice it cannot change that decision later (i.e. link-nondeterminism is resolved before chain-walking continues).

3.1.3 Retiring VIRTIO Observations

When the Device is concluded with its use of a descriptor chain it prepares to send it back to the Driver by filling out a used entry with information about the chain. Once the used index field is written by the Device, all of the chains which have been batched for return to the Driver are returned. It is at this point that the VIRTIO Queue operational model handles retiring related observations.

In particular, the unique Chain IDs associated with each chain track the provenance of the constituent observations, namely, which available ring entry first exposed that chain. In order to maintain temporal isolation, all observations which derive from a given available ring entry must be simultaneously garbage collected.

4 vSwitch Dataplane

The vSwitch dataplane is implemented in a few thousand lines of C++ code (see Section 8 for more precise figures). It is structured as an instantiation of a generic VIRTIO routing layer (VRL). VRL defines a generic dataplane for VIRTIO devices whose core is the `ports_processor` function:

```
void
Vrl::DataPlane::ports_processor() {
    _shared.stats.wakeups++;
    lock_ports();
    for (auto port = _shared.ports.begin();
         port != _shared.ports.end(); ++port) {
        if (port->acquire()) { //port connected
            process_port(*port);
            port->release();
        }
    }
    unlock_ports();
}
```

The dataplane maintains a ports table which is synchronized with the control plane via fine-grained locks on each port (`port->acquire()`) and a single global lock (`lock_ports()`) protecting the structure of the table (adding/removing ports). The per-port locks follow a protocol that permits the control plane to attempt to disconnect/re-connect a port without acquiring the big lock.

The main loop attempts to acquire each port for exclusive use by the dataplane (`port->acquire()`), calling a `process_port` on each acquired lock. Locks that are in the disconnected state cannot be acquired. The `process_port` function checks whether the port has sent new packets to its transmit VIRTIO queue, and processes the new packets by routing them to the appropriate destination ports.

The `ports_processor` function is embedded in a toplevel dataplane loop that repeatedly (1) waits for any client to signal; and (2) upon wakeup, calls `ports_processor` and a second function `ports_notifier` that signals ports that received new packets in this round.

4.1 Virtual Interface Verification

vSwitch instantiates VRL by implementing a virtual function interface specifying how to route packets and how to interact with ports. The virtual `Router` interface, for example, includes functions for connecting and disconnecting ports, for routing packets, for retrieving the next packet from a port, for returning packets to ports once processed, and for completing the “send” of a packet to a given destination port:

```
class Vrl::Router {
public:
  Router() = default;
  virtual ~Router() = default;
  virtual Errno route(Vrl::Port &inp,
    Vrl::Packet &pkt, VrlPortsList &outps)=0;
  virtual Errno connect(Port &port,
    void *data) = 0;
  virtual Errno disconnect(Port &port) = 0;
  virtual Vrl::Packet *pkt(Vrl::Port &port)=0;
  virtual void return_pkt(Vrl::Port &port,
    Vrl::Packet &pkt) = 0;
  virtual Errno send(Vrl::Port &inp_port,
    Vrl::Port &outp_port, Vrl::Packet &pkt)=0;
};
```

The types `Vrl::Packet` and `Vrl::Port` are abstract classes that are implemented by vSwitch, each with their own abstract interfaces.

The workflow of a client of `Vrl::Router` is to receive a packet, call `route` to determine its recipient destination ports, route the packet to the destination ports, and then return the packet to its source port:

```
Vrl::Port &p; ...
/*1. Receive a packet from port p.*/
Vrl::Packet *pkt = _router.pkt(p); ...
/*2. Call 'route' to discover the list of
   recipient ports.*/
VrlPortsList recipients;
_router.route(p, *pkt, recipients); ...
/*3. Send the packet to each recipient.*/
for (auto &recip : recipients) {
  auto err = _router.send(p, recip, *pkt);
  ... }
/*4. Return the packet to the port.*/
_router.return_pkt(p, *pkt);
```

The classes `Vrl::Packet` and `Vrl::Port` have abstract interfaces and protocols as well.

For each abstract class in VRL, we define its interface as a series of specifications in separation logic. Figure 7 gives the abstraction specification of the `route` function, for example. The specification is a bit involved but at high level it:

- associates to each input argument the appropriate abstract representation predicate to each input parameter of the function, e.g., ownership of `Vrl_Port_R` is attached to the input port `p_src`; and
- in the postcondition, returns evidence in the case when `err = Errno.NONE` that the correct list of destination ports has been stored in the output reference parameter `p_outps`

(Payloads (port t) pd dstp dst dstt).

The predicate `Vr1.Port.Map.data_portsP` holds ownership of most data associated to a given list of ports. In the `err = Errno.NONE` case, `route` returns this data for the destination port pointers stored in the output parameter list `p_outps`.

4.2 Specifying External Events

To specify that the C++ implementation of vSwitch refines its operational model, we attach the code to the events generated by the model, identifying external functions in the C++ implementation that form the “application boundary” between the application and the outside world. These functions receive specifications that explain each function’s effect on the model.

Consider the function that writes a single byte to the driver’s used ring. In C++, its prototype might be:

```
Errno write_used(unsigned int idx, char c);
```

where `Errno` gives common error codes like `NONE`, `INVAL`, etc. The specification of this function ties it to the `write_used` event by asking the caller to produce a *requester*, a form of Iris atomic update that proves safety of the `write_used` step and that the step implies postcondition Q .

```
\arg{idx} ‘‘idx’’ (Vint idx)
\arg{c} ‘‘c’’ (Vchar c)
\pre{ $\gamma$  Q}
  requester  $\gamma$  (write_used (buf idx) c) Q
\post{ret}[Verrno ret]
  if (bool_decide (ret == NONE)) then Q
  else  $\llbracket idx \geq \gamma.tx.length \rrbracket$  (*ret == INVAL*)
Errno write_used(...);
```

By parametricity, the implementation of `write_used` proves Q only by discharging the requester provided by the function’s caller (Q is forall quantified by `\pre{ $\gamma$  Q}`).

Internally, a *requester* is an atomic update operating on ghost assertion `AuthSet.frag`, which declares the current set of possible states of the model (following a typical Iris pattern [7], the authoritative half of the assertion, `AuthSet.auth`, is typically kept in an invariant). To prove a requester, the initiator of an event shows that (1) the event is safe in the current state of the model; and (2) once the step has been taken, the postcondition Q holds on the final state s' .

Informally dual to *requesters* are *committers*, which sit on the other side of an external interface between application proofs. Like requesters, committers are indexed by a set of events `STEP` and postcondition Q . Unlike requesters, committers are assumed by the implementers, or *committers*, of events rather than proved by callers. For example, the `write_used` function – implemented as a single memory write – requests that a memory event `write8 (loc2paddr (buf idx)) c` be issued against the “memory”, understood broadly as the memory hierarchy including caches (we discuss our current limitations wrt. weak memory in Section 7.2). The committer in this case is the memory subsystem itself, which responds to `write8` events with appropriately dual `write8` events, indicating the write has been committed.

4.3 From Models to Ghost State

Reasoning directly from the operational model, as the specifications in the previous section do, can be tedious. More maintainable proofs result from first reflecting the operational model in ghost state and then stating convenience lemmas on top of low level requesters and committers.

```

(** Abstract spec of VRL::Router::route(...) **)
Definition route_spec (this : ptr) : WpSpec_cpp :=
  \arg{p_src}   "src"   (Vref p_src)   (*Vrl::Port@*)
  \arg{p_pkt}   "pkt"   (Vref p_pkt)   (*Vrl::Packet@*)
  \arg{p_outps} "outps" (Vref p_outps) (*VrlPortsList@*)
  \with t tport pd
  \prepost{src q_port} p_src  ↦ Vrl_Port_R t tport (cQp.mut q_port) src
  \prepost
    p_src  ↦ Vrl_Port_DataR t tport pd src
  \pre
    p_pkt  ↦ Vrl_Packet_StateR t tport pd p_src q_port (cQp.mut 1)
    (mkState src UnknownDests)

  \pre
    p_outps ↦ List.StructR structR_ty 1 []
  \pre{ports γ_ports} Vrl.Port.Map.data_portsP
    (port t) (valid_maps t)
    pd q_port 1
    ports γ_ports {[ src := p_src ]}

  \prepost
    this ↦ Vrl_Router_R t pd 1
  \require
    γ_ports !! src = Some tport
  \post{err}[Verrno err]
    if bool_decide (err = Errno.NONE) then
      Exists dstp dst dstt,
        [| src ∉ dst |] ** (*guaranteeing the union below is disjoint*)
        [| NoDup dst |] ** (*guaranteeing list_to_map below is not lossy*)
        (*elementwise mapping from dst to dstt is in γ_ports*)
        [| lists_in_map dst dstt γ_ports |] **
        p_pkt  ↦ Vrl_Packet_StateR t tport pd p_src q_port (cQp.mut 1)
        (mkState src (SelectedDests dstp dst dstt)) **
        p_outps ↦ List.StructR structR_ty 1 dstp **
        Payloads (port t) pd dstp dst dstt **
        Vrl.Port.Map.data_portsP (port t) (valid_maps t) pd q_port 1
        ports γ_ports
        (list_to_map (zip dst dstp) ∪ {[ src := p_src ]})
    else
      p_pkt  ↦ Vrl_Packet_StateR t tport pd p_src q_port (cQp.mut 1)
      (mkState src (SelectedDests [] [] [])) **
      p_outps ↦ List.StructR structR_ty 1 [] **
      Vrl.Port.Map.data_portsP (port t) (valid_maps t) pd q_port 1
      ports γ_ports {[ src := p_src ]}.

(*Packaging the virtual interface to be instantiated by vSwitch*)
Definition _vptrI sym impl spec :=
  let tele := [tele (pd : pd.name) (_ : cQp.t) (_ : TT)] in
  let RouterR : tele -t> Rep := fun pd q t => Vrl.Router.R (impl t) pd q in
  vptrI sym (TT := tele) RouterR spec.
Definition virtualI :=
  _vptrI [%cpp[route_hpp.module] named "Vrl::Router::pkt"] router pkt_spec **
  _vptrI [%cpp[route_hpp.module] named "Vrl::Router::return_pkt"] router
    return_pkt_spec **
  _vptrI [%cpp[route_hpp.module] named "Vrl::Router::send"] router send_spec **
  _vptrI route_name router route_spec.

```

Figure 7: Specification in separation logic of VRL virtual route and packaging of route together with `pkt`, `return_pkt`, and `send` as a virtual interface. The interface is instantiated by `vSwitch` (not shown).

For example, it is much more convenient in application proofs to encapsulate functions that interact with shared memory by specifications that describe the effects of these interactions abstractly. Consider, for example, the C++ function `read_desc_next` which reads the `next` field of a descriptor:

```
Errno
read_desc_next(uint32 idx, uint16& out) {
  if (!idx_valid(idx)) return INVALID;
  out = _descs[idx].read_next();
  return NONE;
}
```

The low-level operational specification of this function asserts that it performs a memory event. The higher-level, more usable specification states the function's effects on ghost state:

```
\arg{idx} ``idx'' (Vint idx)
\arg{out} ``out'' (Vref out)
\pre out ↦ anyR Tu16 1
\prepost{γ q g} chain_generation γ q g
\post{ret}[Verrno ret]
  if (ret == NONE) then
    ∃ next, out ↦ u16R 1 next **
      desc_next γ g idx next
  else out ↦ anyR Tu16 1
```

Predicate `desc_next` asserts that for chain g , the `next` field of descriptor idx was observed to have value $addr$. The generation number lets us make `desc_next` persistent (nonspatial), meaning that the following state would be consistent if we were to call `read_desc_next` twice, observing two different values:

```
desc_next γ g idx 1 ** desc_next γ g idx 2
```

In this way we encode abstractly the set semantics of the operational model.

Generation numbers let us do logical “garbage collection” at the points at which the device observes or initiates buffer transfers. The function

```
Errno conclude_chain(Chain g) {
  write_used(NEXT, g);
  ww_fence();
  return write_used_idx(cur_used_idx()+1);
}
```

signals to the driver that the chain g is no longer in use by the device. Its specification invalidates chain g by consuming the chain generation predicate:

```
\arg{g} ``g'' (Chain g)
\pre{γ} chain_generation γ 1 g
\post{ret}[Verrno ret]
  if (ret == NONE) then emp
  else chain_generation γ 1 g
Errno conclude_chain(Chain g);
```

This pattern requires that `chain_generation` be spatial rather than persistent. Lemmas that make use of persistent `desc_next` predicates require matching `chain_generation` facts. Sharing observations between chains, as allowed by the operational model, is supported by lemmas that duplicate persistent facts at new chain generations when allowed by aliasing. New chain generation facts with fresh numbers g are produced when new chains are received in the available ring.

5 Dataclasses for Network Headers

Some parts of a virtual switch like vSwitch do relatively boring work like parsing and serializing network headers. We developed a specification synthesis technique called *dataclasses* to reduce the manual overhead in such proofs.

VIRTIONET headers. Consider the C++ struct in Figure 8 for serializing and deserializing VIRTIONET headers. We call this class a dataclass because it has no interesting abstraction; its model is simply a record with fields for each of the fields of the struct. In many cases, our dataclass specification tools can construct the appropriate model automatically. Here, however, the specification of VIRTIONET headers is part of our vSwitch model, and therefore trusted so we write it by hand:

```
Record VirtioNet.T :=
{ flags : flag_set.t
; gso_type : VirtioNetGSO.gso
; hdr_len : Z
; gso_size : Z
; checksum_start : Z
; checksum_offset : Z
; num_buffers : Z }.
#[only(lens)] derive T.

#[global] Notation VIRTIO_NET_HEADER_SIZE
:= 12%N.

Definition DEFAULT_HEADER : T :=
{| flags := ∅
; gso_type := VirtioNetGSO.GSO_NONE
; hdr_len := Z.of_N VIRTIO_NET_HEADER_SIZE
; gso_size := 0
; checksum_start := 0
; checksum_offset := 0
; num_buffers := 0 |}.

```

Using dataclasses tools, we automatically generate a representation predicate of the C++ struct `VirtioNetHeader` by the code listing given in the lower half of Figure 8. The code specifies a hand-written model for each substruct appearing in `VirtioNetHeader`. For example, the type `VirtioNetFlags` is mapped to the Coq model `VirtioNetHeaderModel.flag_set.t`. Then, the command `#[rep] Dataclass ‘‘VirtioNetHeader’’` uses the supplied mappings to generate a representation predicate of the complete struct `VirtioNetHeader` listed at the top of Figure 8. Dataclasses also generates appropriate hints for using the generated representation in proofs.

Ethernet headers. We did not use the above mentioned dataclass-based approach for Ethernet headers because their size can be 14 bytes or 18 bytes, depending on a value contained in the header. The 13th and 14th bytes in the Ethernet header indicates whether the Ethernet header contains a four-byte vlan tag¹.

In C++, we implemented the Ethernet header parsing as a class holding a pointer to a constant-sized array of unsigned 16-bit numbers. In order to avoid reading guest-provided memory multiple times, we copy bytes that might containing the Ethernet header once into the array owned by the

¹We don’t currently support Ethernet headers of 22 bytes that contain double vlan tags.


```

// C++ implementation of VIRTIONET headers, following
// the VIRTIO standard (5.1.6 Device Operation, version 1.1)
struct VirtioNetHeader {
  VirtioNetFlags flags{VIRTIO_NET_NO_FLAGS};
  VirtioNetGSO gso_type{VIRTIO_NET_GSO_NONE};
  VirtioNetHeaderSize hdr_len{VIRTIO_NET_HEADER_SIZE};
  Little_endian<uint16> gso_size;
  Little_endian<uint16> checksum_start;
  Little_endian<uint16> checksum_offset;
  Little_endian<uint16> num_buffers;

  VirtioNetHeader() {
    gso_size = Little_endian<uint16>::from_native(0);
    checksum_start = Little_endian<uint16>::from_native(0);
    checksum_offset = Little_endian<uint16>::from_native(0);
    num_buffers = Little_endian<uint16>::from_native(0);
  }
};

static_assert(sizeof(VirtioNetHeader) == VIRTIO_NET_HEADER_SIZE);
static_assert(offsetof(VirtioNetHeader, flags) == 0);
static_assert(offsetof(VirtioNetHeader, gso_type) == 1);
static_assert(offsetof(VirtioNetHeader, hdr_len) == 2);
static_assert(offsetof(VirtioNetHeader, gso_size) == 4);
static_assert(offsetof(VirtioNetHeader, checksum_start) == 6);
static_assert(offsetof(VirtioNetHeader, checksum_offset) == 8);
static_assert(offsetof(VirtioNetHeader, num_buffers) == 10);

(** Connect the hand-written Coq model of VIRTIONET headers
    to the C++ struct above. *)
cpp.rep "VirtioNetFeatures" (fun (ti : biIndex) (_Σ : gFunctors)
  '(_Σ : !cpp_logic ti _Σ)
  (σ : genv) (q : cQp.t) (feat : VirtioNet.feature_set.t) =>
  @primR ti _Σ Σ σ (Tenum "_Z17VirtioNetFeatures") q
  (VirtioNet.feature_set.to_bits feat)).
cpp.rep "VirtioNetGSO" (fun (ti : biIndex) (_Σ : gFunctors) ...
cpp.rep "VirtioNetFlags" (fun (ti : biIndex) (_Σ : gFunctors)
  '(_Σ : !cpp_logic ti _Σ)
  (σ : genv) (q : cQp.t) (flg : VirtioNetHeaderModel.flag_set.t) =>
  @primR ti _Σ Σ σ (Tenum "_Z14VirtioNetFlags") q
  (VirtioNetHeaderModel.flag_set.to_bits flg)).
cpp.rep "VirtioNetHeaderSize" (fun (ti : biIndex) (_Σ : gFunctors) ...
cpp.rep (Tnamed "_Z13Little_endianItE") (fun (ti : biIndex)
  (_Σ : gFunctors) '(_Σ : !cpp_logic ti _Σ)
  (σ : genv) (q : cQp.t) (x : Z) =>
  @Little_endianR ti _Σ Σ σ Unsigned W16 q x).

(** Given the mapping of models to C++ code above, generate the
    representation predicate of VirtioNetHeader. *)
#[rep] Dataclass "VirtioNetHeader".

```

Figure 8: C++ struct for serializing and deserializing VIRTIONET headers, and associated Coq code (excerpted at ...) for attaching the C++ code to a hand-written Coq model. Static asserts ensure that the compiler lays out fields at the offsets expected by our specification of struct memory layout.

vswitch implementation. Since the number of copied bytes depend on the length of the whole packet frame provided by the guest ², the class also holds the number of elements that have been copied from the guest-provided memory.

In the Ethernet header parser class, any read of a u16 goes through a particular private function that asserts index limits and converts endianness if needed. Most reads one field of 16bits, but when the implementation reads a MAC addresses, the Ethernet parser needs to combine three u16 numbers into one number. We chose to implement this feature using a loop, though an unrolled repetition is easier to verify.

The C++ implementation does not check against reading vlan tags from a non-vlan headers. When the caller reads a vlan tag out of a non-vlan header, the caller might read a part of the IP header instead.

Our specification of Ethernet header parser class enforces a protocol prevents such misleading reads. Initially, the caller only gets a resource that allows calling a method called `has_enough_bytes()`. Only after the method returns `true`, the caller is allowed to call `is_vlan()` method. Again, only after `is_vlan()` returns `true`, the caller is allowed to access the vlan tag. If any C++ code tries to access the vlan tag before confirming its existence, the code cannot be verified with our specification of the Ethernet header parser.

The mainline proof of Ethernet header parser methods consists of a few lines each. The exception is a MAC address construction with for-loop, whose proof mentions loop invariants explicitly. Most problems are offloaded to proof hints that the mainline proof automatically finds and uses. The subproblems concern the correspondence between the caller-facing predicates and the array contents. The verification also demands that the C++ implementation returns numbers that correspond to the Ethernet header parsing in Coq.

In the future, we plan to use the size field of the Ethernet header, which sometimes exists (otherwise the same field contains a number indicating the type of the Ethernet header). We have not yet figured out how the C++ language specification allows copying bytes into a potentially uninitialized u16 array, so we are still initializing all elements in the u16 array in the implementation. A verified parser combinator library is desirable, especially one that supports dynamically determined layouts.

6 No Direct Flows

Our vSwitch+VIRTIO model emits low-level events directly on memory. To validate the model, we prove that it satisfies three high-level inductive invariants. These invariants together imply that vSwitch never sends packets between different VLANs, a property we call *no direct flows*:

1. at all times and for every transaction the vSwitch processes, the source and destination ports of the transaction lie in the same VLAN;
2. after initialization, the VLAN of a port never changes;
3. after transactions are created, the VLAN of the source of the transaction (and thus by 1 also that of transaction's destination port) never changes.

We express each of invariants 1, 2, and 3 as `LTS_invariants` in Coq. For example, the invariant that ports maintain VLANs is expressed as:

```
Lemma port_vlans_same cfg :  
  LTS_invariant (VSwitch.lts (cfg:=cfg))
```

²The implementation copies up to 76 bytes into the array, enough for parsing the Ethernet, IP and TCP headers. However, some Ethernet packet frames are shorter than 76 bytes.

```

(fun s => forall oev s',
  VSwitch.step (cfg:=cfg) s oev s' ->
  forall pid port port',
  s.(State.port_map)!!pid = Some port ->
  s'.(State.port_map)!!pid = Some port' ->
  port.(vlan) = port'.(vlan)).

```

LTS_invariant lts P says that P holds at any state reachable by stepping the labeled transition system lts from an initial state. The body of port_vlans_same says that for any such reachable state s , if we take one more step to s' , then any port in s' with port ID pid that is also in s will live in the same VLAN before and after the step.

We express invariant 1 – transactions source and destination ports are the same VLAN – by a second LTS_invariant:

```

Lemma srcdst_same_vlan cfg :
  LTS_invariant (VSwitch.lts (cfg:=cfg))
  (fun s => forall t,
    t ∈ s.(VSwitch.State.transactions) ->
    let γ := s.(VSwitch.State.port_map) in
    (∃ src_port,
      γ!!(t.^_src_port_idx) = Some src_port /\
      src_port.^_vlan = t.^_trx_vlan) /\
    (∀ dst_idx,
      t.^_dst_port_idx = Some dst_idx ->
      ∃ dst_port, γ!!dst_idx = Some dst_port /\
      dst_port.^_vlan = t.^_trx_vlan)).

```

The most important conditions are

- $src_port.^_vlan = t.^_trx_vlan$, the source's VLAN equals the transaction's VLAN tag; and
- $dst_port.^_vlan = t.^_trx_vlan$, the destination port's VLAN equals the transaction's VLAN tag.

Thus the two ports are in the same VLAN. To support this formulation of the invariant, we annotate transactions with the VLAN in which they are created.

```

Record transaction := {
  trx_vlan : vlan_id
; src_port_idx : port_id;
; dst_port_idx : option port_id
; ... }.

```

The operational model likewise changes slightly to add VLAN tags when transactions are first created:

```

| StartProcessingPort src_idx src_port nic s'
  (_ : s.^_transactions = [])
  (_ : (s.^_port_map)!!src_idx = Some src_port)
  (_ : src_port.^_nic = Some nic)
  (_ : s' = s &: _transactions . =
    [new_transaction src_idx src_port.(vlan)])
  (_ : oev = None) :
  step s oev s'

```

This StartProcessingPort rule may run whenever the vSwitch's transaction set is empty ($s.^_transactions = []$), e.g., before any port has been processed or after a given port has been fully processed. The first

transaction for a given `src_port` is `new_transaction src_idx src_port.(vlan)`, where `src_port.(vlan)` is the source port's VLAN.

After a transaction has been created, the vSwitch performs additional actions to read the source packet's header, to determine the destination port (or destination ports in the multicast case), and to forward the packet. The operational model evolves the initial transaction to support these steps, possibly spawning new transactions in the multicast case. Lemma `srcdst_same_vlan` implies that for any given transaction the source and destination ports have the same VLAN, but not that the VLANs of transactions are fixed over time. We prove this property as a separate invariant:

```

Lemma trx_vlans_same cfg :
  LTS_invariant (VSwitch.lts (cfg:=cfg))
  (fun s => forall oev s',
    VSwitch.step (cfg:=cfg) s oev s' ->
    forall t t',
    t ∈ s.(transactions) ->
    t' ∈ s'.(transactions) ->
    t.(trx_vlan) = t'.(trx_vlan)).

```

For any reachable vSwitch state, taking one more step results in a transaction set in which every transaction has the same VLAN as each transaction in the set from before the step. This property does not entail that the vSwitch operates on only one VLAN over its entire runtime, only that once a new transaction has spawned from the empty set by `StartProcessingPort`, every further transaction until the working set is empty again has the same VLAN.

7 Limitations

Parts of the vSwitch formal proof are still in progress. We document these and outline future directions below.

7.1 Operational Model Underspecifies Writes

The formulation of “no direct flows” in Section 6 shows that vSwitch constructs and maintains transactions so that a given transaction never crosses VLANs, but does not directly show that reads and writes follow the transaction discipline.

Reads and writes are governed by the operational semantics rules given in Figure 9. Each rule (`Read`, `Write`) follows the same pattern:

- first, the address of the read or write is validated (e.g., `ofs < φsrc.~VSwitch._chain_size_bytes` expressing that the offset of a read is less than the chain size);
- then, the chain offset is translated to an address (`translate (φdst.~VSwitch._resolve) ofs`); and
- finally, the read or write is performed according to the operational model of VIRTIO (`NIC.step ...`).

A limitation of our current formulation of these rules is that bytes written by `Write` are not formally limited to those read by `Read`. Written bytes follow this discipline in practice up to one exception: the first twelve bytes of the destination packet are a constant VIRTIO header which need not correspond exactly to the VIRTIO header received in the source packet. We are in the process of updating the `Write` rule given above with the following additional side condition to reflect that all written bytes except the VIRTIO header come from the transaction's source packet:

```

| Read ports' trx' byte nic nic' addr' perrs'  $\phi$ src src_port ofs
  (_ : ports !! (trx.^_src_port_idx) = Some src_port)
  (_ : src_port.^_nic = Some nic)
  (gid := nic.(NIC.src_tx).(Virtio.Queue.State.vq_config)
    .(Virtio.Queue.vq_guest_id))
  (_ : (trx.^_read_bytes) !! ofs = None)
  (_ : forall size,
    trx.^VSwitch._transaction_size_in_bytes = Some size -> ofs < size)
  (_ : trx.^_src_resolve = Some  $\phi$ src)
  (_ : ofs <  $\phi$ src.^VSwitch._chain_size_bytes)
  (_ : let '(perrs, addr) := translate ( $\phi$ src.^VSwitch._resolve) ofs in
    perrs' = perrs
    /\ addr' = addr
    /\ NIC.step nic
    (Some (NIC.SrcTx (MemRead gid addr' 8(*bits*) byte)) nic'))
  (_ : ports' = alter (_nic .= Some nic') (trx.^_src_port_idx) ports)
  (_ : trx' = trx &: (_read_bytes %= <[ ofs := byte ]>)
    &: (_err .= (SrcErr <$> perrs'))))
  (_ : oev = Some (Event.ByteRead gid addr' byte)) :
step ports trx oev ports' trx'

| Write ports' trx' byte nic nic' addr' perrs' dst_idx dst  $\phi$ dst ofs
  (_ : dst.^_nic = Some nic)
  (gid := nic.(NIC.dst_rx).(Virtio.Queue.State.vq_config)
    .(Virtio.Queue.vq_guest_id))
  (_ : (trx.^_written_bytes) !! ofs = None)
  (_ : forall size,
    trx.^VSwitch._transaction_size_in_bytes = Some size -> ofs < size)
  (_ : trx.^_dst_port_idx = Some dst_idx)
  (_ : ports !! dst_idx = Some dst)
  (_ : trx.^_dst_resolve = Some  $\phi$ dst)
  (_ : ofs <  $\phi$ dst.^VSwitch._chain_size_bytes)
  (_ : let '(perrs, addr) := translate ( $\phi$ dst.^VSwitch._resolve) ofs in
    perrs' = perrs
    /\ addr' = addr
    /\ NIC.step nic
    (Some (NIC.DstRx (MemWrite gid addr' 8(*bits*) byte)) nic'))
  (_ : ports' = alter (_nic .= Some nic') dst_idx ports)
  (_ : trx' = trx &: (_written_bytes %= <[ ofs := byte ]>)
    &: (_err .= (SrcErr <$> perrs'))))
  (_ : oev = Some (Event.ByteWrite gid addr' byte)) :
step ports trx oev ports' trx'

```

Figure 9: Rules of the vSwitch operational semantics governing memory reads and writes

```
(_ : if bool_decide (ofs >= 12)
  then (trx.^_read_bytes)!!ofs = Some byte
  else virtio_net_header_byte ofs byte)
```

The condition states that for bytes at offsets greater than or equal 12, the `byte` being written to the destination port equals the one read from the source port (`trx.^_read_bytes`) !! `ofs = Some byte`). For bytes in range $[0, 12)$, `virtio_net_header_byte ofs byte` states that the `byte` is the one corresponding to offset `ofs` of the constant VIRTIO net header that vSwitch always writes to destination ports.

7.2 Weak Memory

The implementations of vSwitch and VIRTIO are weak memory-aware but our specifications and proofs currently ignore weak memory. We have done some experimental work in the specifications and proofs of VIRTIO to support weak memory reasoning using modalities in the style of FSL [4]. But more work is required to characterize the operational meaning of these modalities in the context of our VIRTIO and vSwitch models.

7.3 Dataplane Admits

File `README-admits.md` in the formal development documents admits (unproved assumptions) in our proof of the vSwitch dataplane. These admits largely fall into the following classes:

- C++ semantics admits related to virtual destructors, to the construction of array objects when the array size is not static, to byte-level views of POD (plain old data) classes, and to the semantics of unions;
- admits introduced when the vSwitch dataplane was refactored to support virtual IOMMUs in BlueRock’s upstream hypervisor product (AsyncControl);
- admits related to ghost state and to the instantiation of a part of the VRL virtual interface.

8 Evaluation

This section evaluates the performance of the vSwitch dataplane and the complexity of the associated specification and proof effort.

8.1 Performance

The throughput of a network switch like vSwitch depends on the size of packets; relatively more work is spent processing network headers when packet payloads are small, yielding lower throughput. For vSwitch, we have found that on a board like the NUC11, these throughputs range from about 2Gbps (small packets) to about 14Gbps (large packets) as measured by `iperf3`.

Figure 10 shows the results of two `iperf3` runs, separating transmit (Tx) throughput from receive (Rx) throughput. To restrict `iperf3` to small packets, we use option `-M 200` (“TCP/MCTP maximum segment size”). Time to transmit (`-t`) is set to 60. Mean transmit throughput over four trials on small packets is 2,638 Mbps (both Tx and Rx) and on large packets 14,675 Mbps (Tx) and 14,674 Mbps (Rx). Confidence intervals for $\alpha = 0.05$ are visible in the chart but small.

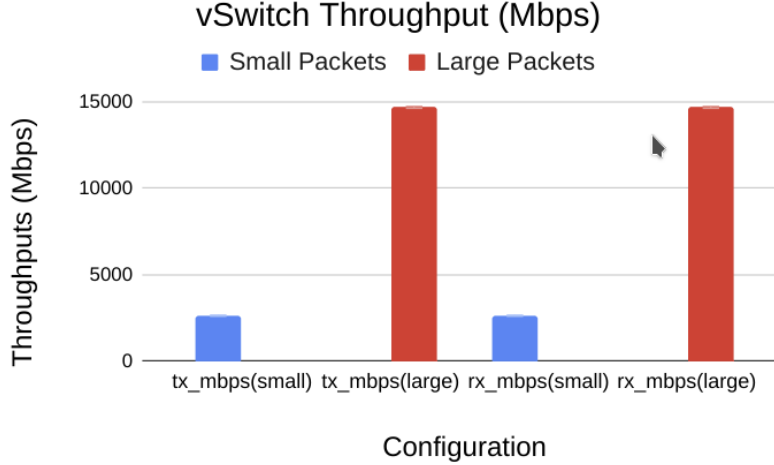


Figure 10: vSwitch transmit (Tx) and receive (Rx) throughput in Mbps as measured by `iperf3` on small and large packets on a NUC11

8.2 Proof Effort

Appendix A includes line counts of C++ and Coq code in the vSwitch development. Parts of the Coq code can be refactored to upstream lemmas out of the vSwitch development into libraries (see, e.g., files like `upstream/lemmata.v`) so we cannot yet definitively characterize proof effort as `#lines of Coq / #lines of C++`. Also, the C++ line counts of vSwitch includes parts of vSwitch as detailed in the appendix that were either out of scope of the verification (e.g., `lib/acl`) or for which the verification is still a work in progress (vSwitch control plane and `src/main.cpp`).

With these caveats, note that vSwitch with VIRTIO and VRL (Virtqueue Routing Library) is 7,386 lines of C++ code excluding comments and blank lines. The Coq development is currently 67,860 lines of code excluding comments and blank lines. This is an approximately 10× proof-to-code ratio.

9 Related Work

The most closely related work is on program logics for robust safety [5], which we detail below. There is additional related work on program logics for imperative languages like RefinedC [11], Diaframe [9], and VST [3], among others. Other projects such as Everest [2] have verified parts of network stacks like TLS. Work on multi-language semantics [10] and game semantics for multi-language systems [6, 8] is related to the model refinement proofs we discuss in Section 6.

10 Conclusion

This report presents protocol completion, a set of guiding principles for formulating specifications and proofs of robust safety over shared-memory protocols, and applies protocol completion to the verification of a VIRTIO-based virtual Ethernet switch.

References

- [1] Private communication.

- [2] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, Catalin Hritcu, Samin Ishtiaq, Markulf Kohlweiss, Rustan Leino, Jay Lorch, Kenji Mailard, Jianyang Pan, Bryan Parno, Jonathan Protzenko, Tahina Ramananandro, Ashay Rane, Aseem Rastogi, Nikhil Swamy, Laure Thompson, Peng Wang, Santiago Zanella-Béguelin, and Jean-Karim Zinzindohoué. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In Benjamin S. Lerner, Rastislav Bodík, and Shriram Krishnamurthi, editors, *2nd Summit on Advances in Programming Languages (SNAPL 2017)*, volume 71 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 1:1–1:12, Dagstuhl, Germany, 2017. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [3] Qinxiang Cao, Lennart Beringer, Samuel Gruetter, Josiah Dodds, and Andrew W. Appel. Vst-floyd: A separation logic tool to verify correctness of C programs. *J. Autom. Reason.*, 61(1-4):367–422, 2018.
- [4] Marko Doko and Viktor Vafeiadis. A program logic for C11 memory fences. In *Verification, Model Checking, and Abstract Interpretation - 17th International Conference, VMCAI 2016, St. Petersburg, FL, USA, January 17-19, 2016. Proceedings*, volume 9583 of *Lecture Notes in Computer Science*, pages 413–430. Springer, 2016.
- [5] Andrew D. Gordon and Alan Jeffrey. Authenticity by typing for security protocols. In *14th IEEE Computer Security Foundations Workshop (CSFW-14 2001), 11-13 June 2001, Cape Breton, Nova Scotia, Canada*, pages 145–159. IEEE Computer Society, 2001.
- [6] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. Certikos: an extensible architecture for building certified concurrent os kernels. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation, OSDI’16*, page 653–669, USA, 2016. USENIX Association.
- [7] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.*, 28:e20, 2018.
- [8] Jérémie Koenig and Zhong Shao. Refinement-based game semantics for certified abstraction layers. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’20*, page 633–647, New York, NY, USA, 2020. Association for Computing Machinery.
- [9] Ike Mulder, Robbert Krebbers, and Herman Geuvers. Diaframe: automated verification of fine-grained concurrent programs in Iris. In Ranjit Jhala and Isil Dillig, editors, *PLDI ’22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, pages 809–824, 2022.
- [10] James T. Perconti and Amal Ahmed. Verifying an open compiler using multi-language semantics. In *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, page 128–148, Berlin, Heidelberg, 2014. Springer-Verlag.
- [11] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. Refinedc: automating the foundational verification of C code with refined ownership types. In Stephen N. Freund and Eran Yahav, editors, *PLDI ’21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, pages 158–174. ACM, 2021.

- [12] David Swasey, Deepak Garg, and Derek Dreyer. Robust and compositional verification of object capability patterns. *Proc. ACM Program. Lang.*, 1(OOPSLA):89:1–89:26, 2017.
- [13] The BlueRock Security Formal Methods Team. The BRiCk C++ Program Logic, 2025.
- [14] Michael S. Tsirkin and Cornelia Huck. *Virtual I/O Device (VIRTIO)*. OASIS Virtual I/O Device (VIRTIO) Technical Committee, April 2019.

Appendix A: Line Counts

This appendix gives the verbatim line count output of `cloc` on the `vswitch` and `devices/virtio_base` directories. We separate C++ files (`.hpp`, `.cpp`) from Coq files (`.v`) using the following distinct invocations of `cloc`:

```
// Count C++ lines
find ./ -type f \( -iname \*.cpp -o -iname \*.hpp \) | xargs cloc --by-file
// Count Coq lines
find ./ -type f \( -iname \*.v \) | xargs cloc --by-file
```

The count of C++ files in `vSwitch` includes parts of `vSwitch` that were out of scope of the verification (`lib/vswitch_cli`, `lib/acl`, `lib/vsmp`, `lib/vswitch_manager`).

vSwitch C++ Files

github.com/AlDanial/cloc v 2.00 T=0.03 s (1605.8 files/s, 236499.9 lines/s)

File	blank	comment	code
./lib/vswitch_cli/src/commands.cpp	181	30	787
./lib/vswitch/src/vswitch.cpp	102	59	483
./lib/forwarding/src/forwarding_plane.cpp	89	76	469
./lib/vswitch_config/src/bson_config.cpp	69	115	259
./lib/vsmp/include/vsmp/msg_queue.hpp	51	31	210
./lib/acl/src/acl.cpp	36	77	164
./lib/port/include/port/port.hpp	55	25	147
./lib/acl/src/parser.cpp	34	59	143
./lib/vswitch_manager/src/vswitch_manager.cpp	28	8	143
./lib/protocol/include/protocol/ethernet.hpp	22	13	136
./src/main.cpp	30	8	134
./lib/vswitch/include/vswitch/vswitch.hpp	36	19	133
./lib/acl/include/acl/match_field_types.hpp	32	18	113
./lib/acl/include/acl/matcher.hpp	22	12	98
./lib/vswitch_cli/include/vswitch_cli/commands.hpp	17	7	90
./lib/protocol/include/protocol/ipv4.hpp	24	27	89
./lib/util/include/util/profiler.hpp	16	9	78
./lib/protocol/include/protocol/virtio_net.hpp	9	15	74
./lib/vswitch_manager/src/vswitch_acl.cpp	14	4	72
./lib/protocol/include/protocol/ipv6.hpp	24	23	71
./lib/forwarding/include/forwarding/forwarding_plane.hpp	31	54	70
./lib/vswitch_cli/src/vswitch_cli.cpp	21	11	68
./lib/forwarding/include/forwarding/forwarding_table.hpp	14	15	52
./lib/vswitch_config/include/vswitch_config/bson_config.hpp	17	34	50
./lib/acl/include/acl/acl.hpp	14	13	47
./lib/vswitch_cli/include/vswitch_cli/vswitch_cli.hpp	11	4	42
./lib/acl/include/acl/flow_list.hpp	15	32	39
./lib/acl/include/acl/outcome.hpp	10	9	37
./lib/vswitch_manager/src/management.cpp	9	4	37
./lib/acl/include/acl/flow_entry.hpp	12	18	35
./lib/vswitch_manager/include/vswitch_manager/vswitch_manager.hpp	12	13	34
./lib/vsmp/include/vsmp/event.hpp	11	19	32
./lib/acl/include/acl/actions.hpp	14	16	30
./lib/protocol/include/protocol/ip.hpp	3	8	25
./lib/util/include/util/trace.hpp	5	4	22
./lib/vsmp/include/vsmp/vsmp.hpp	5	15	22
./lib/protocol/include/protocol/tcp.hpp	7	4	21
./lib/vsmp/include/vsmp/request.hpp	6	17	21
./lib/vswitch/include/vswitch/vswitch_config.hpp	5	4	21
./lib/protocol/include/protocol/udp.hpp	7	4	20
./lib/acl/include/acl/packet_field.hpp	9	9	18
./lib/vsmp/include/vsmp/response.hpp	5	10	14
./lib/vswitch/src/test_vswitch.cpp	2	0	9

./lib/acl/include/acl/parser.hpp	3	4	7
./lib/acl/include/acl/acl_config.hpp	2	4	6
./include/config.hpp	0	1	1
SUM:	1141	961	4673

vSwitch Coq Files

github.com/AlDanial/cloc v 2.00 T=0.20 s (1070.7 files/s, 262008.9 lines/s)

File	blank	comment	code
./proof/upstream/lemmata.v	362	187	5551
./lib/forwarding/proof/forwarding_table_hpp/hints.v	124	39	3359
./lib/protocol/proof/ethernet_hpp/hints.v	110	22	1697
./proof/ghost.v	190	135	1605
./lib/forwarding/proof/transaction_protocol/hints.v	117	149	1214
./lib/vsmp/proof/msg_queue_hpp/proof.v	212	108	1140
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/buffer/copy.v	112	152	1090
./proof/model/vswitch/lemmas.v	36	26	1002
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/buffer/conclude_chain_use.v	78	58	930
./proof/model/ethernet.v	53	57	902
./lib/forwarding/proof/forwarding_plane_cpp/hints/route.v	113	22	876
./proof/model/virtionet_header.v	69	20	865
./lib/vswitch/proof/vswitch_hpp/defs.v	107	43	773
./lib/forwarding/proof/forwarding_plane_hpp/defs.v	124	118	726
./lib/forwarding/proof/forwarding_plane_cpp/hints/private_hints3.v	75	27	640
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/buffer/get_set_byte_requesters.v	54	72	633
./lib/forwarding/proof/forwarding_plane_hpp/hints/extract_frame_header.v	44	34	633
./lib/forwarding/proof/transaction_protocol/defs.v	74	188	608
./lib/vswitch/proof/vswitch_cpp/proof.v	95	34	608
./lib/forwarding/proof/forwarding_plane_hpp/spec.v	65	108	599
./lib/forwarding/proof/ghost_hints.v	89	70	598
./lib/forwarding/proof/forwarding_plane_hpp/hints/misc_hints.v	67	19	572
./lib/vsmp/proof/msg_queue_hpp/defs.v	110	24	566
./lib/port/proof/port/defs.v	109	96	552
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtual_interface.v	82	25	529
./lib/forwarding/proof/forwarding_plane_cpp/hints/flood.v	134	20	514
./lib/port/proof/port/spec.v	77	40	497
./lib/forwarding/proof/forwarding_plane_cpp/hints.v	131	158	493
./lib/port/proof/port/proof.v	62	118	447
./lib/vswitch/proof/vswitch_cpp/spec.v	36	73	445
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/util.v	23	22	443
./lib/vsmp/proof/msg_queue_hpp/spec.v	65	16	439
./proof/upstream/hints.v	36	15	423
./lib/port/proof/port/hints.v	62	38	391
./lib/forwarding/proof/forwarding_plane_cpp_proof.v	45	504	370
./proof/upstream/lts_inv.v	46	14	331

./lib/forwarding/proof/forwarding_plane_hpp/hints/virtio_net_header_hints.v	36	18	323
./proof/model/vswitch/model.v	46	142	309
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/buffer/walk_chain.v	25	16	280
./lib/forwarding/proof/forwarding_plane_cpp/hints/copy_frame.v	49	33	278
./lib/util/proof/profiler_hpp_spec.v	45	14	266
./lib/forwarding/proof/forwarding_plane_cpp/proof/misc.v	28	26	250
./lib/forwarding/proof/forwarding_plane_cpp/proof/route.v	52	69	242
./lib/forwarding/proof/forwarding_table_hpp/defs.v	52	41	226
./lib/vswitch/proof/create_forwarding_plane.v	32	34	215
./lib/forwarding/proof/forwarding_plane_cpp/proof/flood.v	37	69	210
./proof/model/vswitch/defs.v	43	29	209
./lib/protocol/proof/ethernet_hpp/proof.v	21	22	205
./proof/model/util.v	25	15	205
./lib/forwarding/proof/forwarding_plane_cpp/proof/virtual_interface.v	20	15	192
./lib/forwarding/proof/forwarding_plane_hpp/hints.v	48	33	189
./lib/forwarding/proof/forwarding_table_hpp/proof.v	14	29	155
./lib/port/proof/port/upstream.v	32	12	153
./lib/protocol/proof/ethernet_hpp/spec.v	23	16	153
./lib/forwarding/proof/forwarding_plane_cpp/hints/cpp_hints.v	36	19	151
./proof/upstream/map_seqN.v	19	21	151
./lib/forwarding/proof/forwarding_plane_cpp/hints/data_portsP_hints.v	21	10	147
./lib/forwarding/proof/forwarding_plane_cpp/proof.v	23	32	144
./lib/forwarding/proof/forwarding_plane_cpp/proof/copy_frame.v	17	12	135
./lib/vswitch/proof/initialize_dataplane.v	19	34	134
./lib/forwarding/proof/forwarding_plane_hpp/hints/vrl_hints.v	8	9	130
./lib/acl/proof/flow_list_hpp_spec.v	25	37	127
./lib/vswitch/proof/vswitch_hpp_spec2.v	13	14	121
./lib/vsmp/proof/msg_queue_hpp/hints.v	24	5	120
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/devq/used_event_notify.v	15	8	117
./lib/acl/proof/outcome_hpp_spec.v	23	9	116
./lib/forwarding/proof/forwarding_plane_cpp/proof/extract_frame_header.v	10	37	112
./lib/forwarding/proof/forwarding_plane_hpp/hints/misc.v	18	5	110
./lib/acl/proof/matcher_hpp_spec.v	36	67	105
./lib/vswitch/proof/vswitch_hpp/spec.v	23	106	104
./lib/forwarding/proof/forwarding_plane_hpp/hints/drop.v	10	10	102
./lib/forwarding/proof/forwarding_plane_hpp/hints/set_virtio_net_header.v	17	29	92
./lib/forwarding/proof/forwarding_plane_hpp/defs.v	13	16	89
./lib/forwarding/proof/forwarding_plane_hpp/spec.v	25	28	87
./lib/forwarding/proof/forwarding_plane_hpp/proof/set_virtio_net_header.v	13	24	85
./lib/acl/proof/actions_hpp_spec.v	22	5	84
./lib/forwarding/proof/forwarding_plane_hpp/proof/packet_ctor.v	13	14	82
./lib/forwarding/proof/forwarding_plane_hpp/hints/callback_impl.v	21	8	81
./lib/forwarding/proof/forwarding_table_hpp/spec.v	14	29	81
./lib/vswitch/proof/vswitch_hpp/defs.v	21	20	76
./lib/forwarding/proof/forwarding_plane_hpp/hints/vlan_groups_hints.v	18	23	71
./lib/vswitch/proof/vswitch_config_hpp/defs.v	11	14	70
./lib/protocol/proof/ethernet_hpp/defs.v	13	17	68
./proof/upstream/gmap_lemmas.v	11	10	68
./lib/forwarding/proof/forwarding_plane_hpp_spec.v	23	13	66

./proof/elpi/learn.v	43	102	66
./lib/forwarding/proof/forwarding_plane_cpp/hints/vqueue_interface.v	14	8	65
./proof/typeclasses.v	11	12	53
./lib/forwarding/proof/forwarding_plane_cpp/proof/pkt.v	8	32	52
./lib/protocol/proof/tcp_hpp_spec.v	16	17	47
./lib/vswitch/proof/vswitch_hpp/hints.v	5	13	44
./lib/acl/proof/acl_hpp_spec.v	25	39	43
./lib/protocol/proof/udp_hpp_spec.v	15	17	43
./proof/upstream/hyp_tagging.v	8	5	43
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/set_virtio_net_header.v	15	14	40
./lib/protocol/proof/virtio_net_hpp/hints.v	9	15	38
./lib/vsmp/proof/msg_queue_hpp/prelude.v	4	5	36
./proof/upstream/record_equality.v	15	20	35
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/virtual_interface.v	12	8	34
./lib/forwarding/proof/forwarding_plane_cpp/proof/drop.v	6	7	34
./proof/prelude/common.v	11	19	33
./lib/acl/proof/match_field_types_hpp_proof.v	12	38	29
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/misc.v	6	8	29
./lib/protocol/proof/virtio_net_hpp/proof.v	10	12	29
./lib/acl/proof/packet_field_hpp_spec.v	19	40	28
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/route.v	10	6	28
./lib/vswitch/proof/vswitch_config_hpp/spec.v	7	12	28
./lib/forwarding/proof/forwarding_table_hpp/upstream.v	8	8	27
./lib/forwarding/proof/forwarding_plane_cpp/hints/pkt.v	12	7	26
./lib/forwarding/proof/forwarding_plane_cpp/prelude/hints/copy_frame.v	7	5	25
./lib/acl/proof/flow_entry_hpp_spec.v	19	71	23
./lib/protocol/proof/ipv6_hpp_spec.v	9	5	23
./lib/util/proof/trace_hpp_spec.v	16	67	23
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/flood.v	13	5	21
./lib/protocol/proof/ipv4_hpp_spec.v	9	5	21
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/extract_frame_header.v	4	8	20
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/pkt.v	6	5	20
./lib/vswitch/proof/vswitch_hpp/prelude/spec.v	6	14	20
./proof/prelude/defs.v	11	24	20
./proof/prelude/spec.v	12	10	20
./lib/forwarding/proof/forwarding_plane_hpp/prelude/defs.v	10	17	19
./lib/port/proof/port/prelude/defs.v	6	11	19
./proof/prelude/hints.v	12	13	19
./lib/forwarding/proof/forwarding_plane_cpp/prelude/hints.v	4	13	18
./lib/forwarding/proof/forwarding_plane_cpp/proof/send.v	3	5	18
./lib/protocol/proof/virtio_net_hpp/spec.v	6	12	16
./lib/vswitch/proof/vswitch_vrl.v	3	6	16
./lib/acl/proof/parser_hpp_spec.v	10	5	15
./lib/protocol/proof/virtio_net_hpp/defs.v	4	12	14
./lib/vswitch/proof/vswitch_cpp/prelude/spec.v	4	13	14
./lib/forwarding/proof/forwarding_plane_cpp/hints/forwarding_table_private_hints.v	3	6	13
./lib/forwarding/proof/forwarding_plane_cpp/prelude/ghost.v	2	0	13
./lib/forwarding/proof/forwarding_plane_cpp/prelude/model.v	2	0	13
./lib/forwarding/proof/forwarding_plane_hpp/prelude/ghost.v	2	0	13

./lib/forwarding/proof/forwarding_plane_hpp/prelude/model.v	2	0	13
./lib/forwarding/proof/forwarding_plane_hpp/prelude/proof.v	2	0	13
./lib/protocol/proof/ethernet_hpp/prelude/model.v	1	0	13
./proof/prelude/ghost.v	5	11	13
./lib/forwarding/proof/forwarding_plane_cpp/model.v	2	0	12
./lib/forwarding/proof/forwarding_plane_hpp/ghost.v	2	0	12
./lib/forwarding/proof/forwarding_plane_hpp/model.v	2	0	12
./lib/forwarding/proof/transaction_protocol/prelude/defs.v	10	14	12
./lib/vswitch/proof/vswitch_hpp/prelude/defs.v	4	14	12
./lib/forwarding/proof/forwarding_plane_cpp/prelude/defs.v	7	13	11
./lib/forwarding/proof/forwarding_plane_hpp/prelude/spec.v	5	13	11
./lib/forwarding/proof/forwarding_plane_cpp/prelude/spec.v	6	14	10
./proof/model/nic.v	2	8	10
./proof/prelude/model.v	4	17	9
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters.v	4	10	8
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof/copy_frame.v	5	5	8
./lib/forwarding/proof/forwarding_table_hpp/prelude/defs.v	4	16	8
./lib/protocol/proof/virtio_net_hpp/model.v	3	12	8
./lib/acl/proof/acl_config_hpp_spec.v	5	5	7
./lib/acl/proof/match_field_types_hpp_spec.v	20	100	7
./lib/forwarding/proof/transaction_protocol/prelude/hints.v	6	13	7
./lib/port/proof/port/prelude/spec.v	2	6	7
./lib/protocol/proof/ethernet_hpp/prelude/proof.v	5	13	7
./lib/vswitch/proof/vswitch_cpp/prelude/defs.v	3	13	7
./lib/vswitch/proof/vswitch_cpp/prelude/proof.v	2	13	7
./lib/forwarding/proof/forwarding_plane_hpp/prelude/hints.v	5	13	6
./lib/port/proof/port/prelude/proof.v	2	5	6
./lib/protocol/proof/ethernet_hpp/prelude/defs.v	6	15	6
./lib/protocol/proof/ip_hpp_spec.v	3	5	6
./lib/vswitch/proof/vswitch_config_hpp/prelude/defs.v	4	14	6
./lib/vswitch/proof/vswitch_hpp_proof.v	0	0	6
./lib/vswitch_config/proof/bson_config_hpp_spec.v	3	5	6
./lib/port/proof/port/prelude/hints.v	3	5	5
./lib/protocol/proof/ethernet_hpp/model.v	4	12	5
./lib/protocol/proof/ethernet_hpp/prelude/hints.v	3	13	5
./lib/protocol/proof/ethernet_hpp_spec.v	1	5	5
./lib/protocol/proof/virtio_net_hpp_spec.v	1	5	5
./proof/model.v	1	6	5
./proof/model/vswitch/prelude/lemmas.v	5	15	5
./proof/model/vswitch/prelude/model.v	4	13	5
./lib/protocol/proof/virtio_net_hpp/prelude/hints.v	3	13	4
./lib/protocol/proof/virtio_net_hpp/prelude/model.v	4	13	4
./lib/protocol/proof/virtio_net_hpp/prelude/proof.v	3	13	4
./lib/vswitch/proof/vswitch_config_hpp/prelude/spec.v	3	14	4
./proof/model/vswitch/prelude/defs.v	4	16	4
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/buffer.v	0	5	3
./lib/forwarding/proof/forwarding_plane_cpp/prelude/hints/drop.v	1	5	3
./lib/forwarding/proof/forwarding_table_hpp/prelude/hints.v	3	13	3
./lib/forwarding/proof/forwarding_table_hpp/prelude/proof.v	3	13	3

./lib/protocol/proof/virtio_net_hpp/prelude/defs.v	3	13	3
./lib/protocol/proof/virtio_net_hpp/prelude/spec.v	3	13	3
./lib/vswitch/proof/vswitch_config_hpp/prelude/proof.v	2	13	3
./lib/vswitch/proof/vswitch_hpp/prelude/hints.v	2	13	3
./lib/vswitch/proof/vswitch_hpp/prelude/proof.v	2	13	3
./proof/model/vswitch.v	1	6	3
./proof/prelude/proof.v	1	9	3
./lib/forwarding/proof/forwarding_table_hpp/prelude/spec.v	2	13	2
./lib/forwarding/proof/transaction_protocol.v	0	5	2
./lib/port/proof/port/prelude/ghost.v	1	6	2
./lib/protocol/proof/ethernet_hpp/prelude/spec.v	3	13	2
./lib/protocol/proof/ethernet_hpp_proof.v	1	6	2
./lib/protocol/proof/virtio_net_hpp/prelude/ghost.v	2	13	2
./lib/vswitch/proof/vswitch_config_hpp/prelude/ghost.v	2	13	2
./lib/vswitch/proof/vswitch_config_hpp/prelude/hints.v	2	13	2
./lib/vswitch/proof/vswitch_config_hpp_spec.v	0	5	2
./lib/vswitch/proof/vswitch_cpp/prelude/ghost.v	2	13	2
./lib/vswitch/proof/vswitch_cpp/prelude/hints.v	2	13	2
./lib/vswitch/proof/vswitch_hpp/prelude/ghost.v	2	13	2
./lib/vswitch/proof/vswitch_hpp_spec.v	1	5	2
./lib/forwarding/proof/forwarding_plane_cpp/hints/virtio_requesters/devq.v	0	5	1
./lib/forwarding/proof/forwarding_plane_cpp/prelude/proof.v	1	13	1
./lib/port/proof/port/prelude/model.v	0	5	1
./lib/protocol/proof/ethernet_hpp/ghost.v	2	12	1
./lib/protocol/proof/ethernet_hpp/prelude/ghost.v	2	13	1
./lib/protocol/proof/virtio_net_hpp/ghost.v	2	12	1
./lib/protocol/proof/virtio_net_hpp_proof.v	1	5	1
./lib/vswitch/proof/vswitch_config_hpp/hints.v	2	12	1
./lib/vswitch/proof/vswitch_config_hpp/prelude/model.v	2	13	1
./lib/vswitch/proof/vswitch_config_hpp/proof.v	2	12	1
./lib/vswitch/proof/vswitch_cpp/hints.v	2	12	1
./lib/vswitch/proof/vswitch_cpp/prelude/model.v	2	13	1
./lib/vswitch/proof/vswitch_cpp_spec.v	0	5	1
./lib/vswitch/proof/vswitch_hpp/prelude/model.v	2	13	1
./lib/vswitch/proof/vswitch_hpp/proof.v	2	12	1

SUM:	5313	5879	42153

VIRTIO C++ Files

github.com/AlDanial/cloc v 2.00 T=0.01 s (601.7 files/s, 246173.2 lines/s)

File	blank	comment	code

./src/virtio_sg.cpp	197	191	753
./include/model/virtio_sg.hpp	92	174	407
./include/model/virtqueue.hpp	94	166	358
./include/model/virtio_common.hpp	66	14	284

./include/model/virtio_mmio.hpp	27	14	191
./src/virtqueue.cpp	52	125	134
./include/model/virtio.hpp	23	11	79
./include/model/iommu_interface.hpp	21	33	76
./include/model/foreign_ptr.hpp	19	25	56
SUM:	591	753	2338

VIRTIO Coq Files

github.com/AlDanial/cloc v 2.00 T=0.12 s (805.9 files/s, 243608.5 lines/s)

File	blank	comment	code
./proof/virtqueue/requesters.v	222	449	2048
./proof/virtio_sg/spec.v	212	332	1801
./proof/virtqueue/spec.v	237	242	1464
./proof/model/virtq/state/descriptor_table.v	117	150	1439
./proof/virtio_sg/proof/spec.v	149	274	1266
./proof/virtio_sg/defs.v	105	108	946
./proof/virtqueue/ghost.v	100	247	945
./proof/virtio_sg/requesters.v	65	149	920
./proof/virtqueue/defs.v	125	158	885
./proof/virtio_sg/ghost.v	91	213	837
./proof/virtio_sg/proof/buffer/copy/to_sg/TO_UPSTREAM.v	70	104	764
./proof/virtio_sg/hints.v	86	66	700
./proof/model/util.v	81	36	655
./proof/foreign_ptr_spec.v	88	144	501
./proof/virtio_sg/proof/buffer/misc.v	48	45	384
./proof/virtqueue/hints.v	60	43	382
./proof/model/virtq/location.v	44	84	328
./proof/virtio_sg/proof/buffer/conclude_chain_use.v	27	43	323
./proof/model/virtq/step/device/descriptor_table.v	27	131	279
./proof/virtio_sg/proof/buffer/copy/to_sg/sync_impl.v	43	112	268
./proof/virtqueue/proof/devq/send.v	26	28	262
./proof/ghost_common.v	42	74	220
./proof/model/virtq/step/device.v	18	142	214
./proof/model/virtq/state/common.v	29	61	203
./proof/virtqueue/proof/spec.v	30	18	192
./proof/virtio_sg/proof/buffer/async_copy_cookie.v	26	11	179
./proof/virtio_sg/proof/buffer/iterator.v	18	19	174
./proof/model/virtq/step/device/used_ring.v	16	73	172
./proof/virtio_sg/proof/hints.v	17	19	155
./proof/virtio_sg/proof/buffer/copy/to_sg/async_glue.v	26	35	150
./proof/virtio_sg/proof/buffer/copy/to_sg/async_default_impl.v	38	74	141
./proof/virtio_sg/proof/TO_UPSTREAM.v	21	25	138
./proof/virtqueue/proof/queue/misc.v	14	18	136
./proof/model/virtq/state/used_ring.v	27	45	135

./proof/virtqueue/proof/used_entry/setters.v	27	24	130
./proof/model/virtq.v	37	99	111
./proof/virtqueue/proof/is_size_valid.v	13	8	105
./proof/model/virtq/state/available_ring.v	21	54	95
./proof/model/virtq/step/device/available_ring.v	19	99	95
./proof/virtio_sg/proof/buffer/bulk_copier_default.v	13	17	95
./proof/virtqueue/proof/used/setters.v	17	17	94
./proof/virtio_sg/proof/buffer/chain_walking_nop.v	13	12	91
./proof/virtqueue/proof/devq/misc.v	7	7	54
./proof/virtqueue/proof/descriptor/misc.v	8	9	48
./proof/model/virtq/state.v	11	32	47
./proof/virtqueue/proof/available/misc.v	7	10	45
./proof/virtqueue/proof/used/misc.v	7	10	45
./proof/model/virtq/step/driver.v	5	52	43
./proof/virtio_sg/proof/desc_metadata.v	8	8	43
./proof/model/common.v	4	7	35
./proof/prelude/common.v	12	20	30
./proof/virtio_sg/proof/buffer/chain_accessor.v	5	5	28
./proof/virtio_sg/proof/linearized_desc.v	7	5	28
./proof/model/iommu_interface_hpp/hints.v	6	13	23
./proof/virtqueue/proof/used_entry/misc.v	3	7	19
./proof/prelude/hints.v	9	12	16
./proof/virtio_sg/proof/buffer/bulk_copier.v	4	5	16
./proof/virtio_sg/proof/buffer/chain_walking_callback.v	4	5	16
./proof/prelude/defs.v	5	13	14
./proof/upstream.v	1	9	14
./proof/model/iommu_interface_hpp/prelude/model.v	2	0	13
./proof/model/iommu_interface_hpp/defs.v	7	12	12
./proof/virtio_sg/proof/buffer.v	0	5	10
./proof/virtio_sg/proof/deps.v	3	5	10
./proof/virtqueue/proof/deps.v	4	5	10
./proof/prelude/spec.v	3	10	8
./proof/virtqueue/proof/hints.v	3	5	8
./proof/prelude/ghost.v	3	11	7
./proof/prelude/model.v	3	14	7
./proof/iommu_interface_hpp_spec.v	0	0	6
./proof/virtio_sg/proof/buffer/updaters.v	2	5	6
./proof/virtqueue/proof.v	1	5	6
./proof/model/iommu_interface_hpp/prelude/defs.v	2	13	5
./proof/prelude/proof.v	2	10	5
./proof/model/virtq/step.v	1	7	4
./proof/virtio_sg.v	0	8	4
./proof/virtio_sg/proof/buffer/copy/to_sg.v	0	5	4
./proof/virtqueue.v	0	8	4
./proof/virtio_sg/proof.v	1	8	3
./proof/model/iommu_interface_hpp/prelude/hints.v	2	13	2
./proof/model/iommu_interface_hpp/prelude/proof.v	2	13	2
./proof/virtqueue/proof/used.v	0	5	2
./proof/virtqueue/proof/used_entry.v	0	5	2

./proof/model/iommu_interface_hpp/ghost.v	2	12	1
./proof/model/iommu_interface_hpp/model.v	2	12	1
./proof/model/iommu_interface_hpp/prelude/ghost.v	2	13	1
./proof/model/iommu_interface_hpp/prelude/spec.v	2	13	1
./proof/model/iommu_interface_hpp/proof.v	2	12	1
./proof/model/iommu_interface_hpp/spec.v	2	12	1
./proof/virtqueue/proof/TO_UPSTREAM.v	1	6	1
./proof/virtqueue/proof/available.v	0	5	1
./proof/virtqueue/proof/descriptor.v	0	5	1
./proof/virtqueue/proof/devq.v	1	6	1
./proof/virtqueue/proof/queue.v	0	5	1
SUM:	2673	4604	21137

VRL C++ Files

github.com/AlDanial/cloc v 2.00 T=0.01 s (638.9 files/s, 61091.4 lines/s)

File	blank	comment	code
./src/dataplane.cpp	31	69	124
./include/vrl/port.hpp	37	93	105
./include/vrl/dataplane.hpp	14	20	44
./include/vrl/shared.hpp	8	4	33
./include/vrl/packet.hpp	11	13	28
./include/vrl/route.hpp	9	62	20
./include/vrl/stats.hpp	3	10	15
./include/vrl/vrl.hpp	2	4	6
SUM:	115	275	375

VRL Coq Files

github.com/AlDanial/cloc v 2.00 T=0.03 s (786.5 files/s, 241537.4 lines/s)

File	blank	comment	code
./proof/vrl/dataplane_hpp_proof.v	170	64	1146
./proof/vrl/port_hpp_spec.v	231	72	1143
./proof/vrl/shared_hpp_spec.v	70	26	335
./proof/vrl/port_list_hints.v	60	27	328
./proof/vrl/dataplane_hpp_spec.v	48	12	264
./proof/vrl/port_hpp_proof.v	41	30	238
./proof/vrl/route_hpp_spec.v	56	74	232
./proof/TO_UPSTREAM.v	39	13	147
./proof/vrl/port_map_hints.v	84	190	137
./proof/vrl/packet_hpp_spec.v	51	66	130

./proof/vrl/port_list_payload.v	21	5	122
./proof/vrl/packet_hpp_proof.v	12	5	58
./proof/lists_in_map.v	5	5	57
./proof/vrl/port_hpp_hint.v	10	7	57
./proof/guardedR.v	9	5	45
./proof/vrl/shared_hpp_hints.v	8	5	42
./proof/vrl/typeclasses.v	3	6	29
./proof/vrl/route_hpp_proof.v	13	9	23
./proof/vrl/control_plane_spec.v	3	7	19
./proof/vrl/spec_imports.v	4	6	18

SUM:	938	634	4570
